

Collections Framework and Dynamic Lists

CO6 / MODULE 6 / WEEK 11 / LECTURE 1



SMART LOCKER: maintain a parcel register that grows as deliveries arrive

UNIFLOW: maintain a student admission register without a fixed capacity

Dr. N Chaitanya Kumar

Professor in CSE & Coordinator Freshman Engineering Department

Fixed arrays and dynamic collections

The number of parcels or admissions changes during execution, while an array keeps one length.

ARRAY

```
Student[] students = new Student[3]
```

Capacity stays 3

Program tracks the used count

Insertion after capacity requires a larger replacement array

CHOOSE

ARRAYLIST

```
List<Student> students = new ArrayList<>()
```

Logical size grows and shrinks

add and remove express the operation directly

size reports the current element count

Choose a collection when the application must grow, shrink, search, update or remove records during execution.

Collections Framework map



The interface describes the behaviour the program needs; the implementation provides it.

INTERFACE	CORE IDEA	COMMON IMPLEMENTATIONS	APPLICATION QUESTION
List	ordered sequence with indexes	ArrayList	Do duplicates and position matter?
Set	unique elements	HashSet, TreeSet	Must duplicates be rejected?
Map	key mapped to a value	HashMap, TreeMap	Do we retrieve by a unique key?
Queue	first element waiting for service	ArrayDeque	Who should be processed next?

Start with the requirement and choose the interface before choosing a concrete implementation.

Collection and Collections are different

One is an interface in the type hierarchy; the other is a utility class with static algorithms.

Collection<E>

Interface for a group of elements

Parent idea for List, Set and Queue

Examples: add, remove, size

Map belongs to the framework but does not extend Collection

NAME

Collections

Utility class

Provides static methods

Examples: sort, reverse, binarySearch

Used earlier in CO5 with List algorithms

The final letter s changes the meaning: Collection is a type; Collections supplies utility methods.

Creating a typed ArrayList

The interface appears on the left; the concrete implementation appears after new.

DECLARATION AND CREATION

```
import java.util.ArrayList;
import java.util.List;

List<Parcel> parcels = new ArrayList<>();
List<Student> students = new ArrayList<>();

System.out.println(parcels.size()); // 0
System.out.println(parcels.isEmpty()); // true
```

PART	MEANING
List<Parcel>	only Parcel elements
parcels	reference variable
new ArrayList<>()	creates empty object
<> or diamond	compiler infers Parcel

TYPE SAFETY

Adding a Student to parcels becomes a compile-time error rather than a later casting failure.

Typed collections state what kind of element the application permits.

Adding records and preserving order



`add(element)` appends; `add(index, element)` inserts and shifts later elements right.

SMART LOCKER EXAMPLE

```
List<Parcel> parcels = new ArrayList<>();
parcels.add(new Parcel("PKG-101", "Anitha"));
parcels.add(new Parcel("PKG-103", "Ravi"));
parcels.add(1, new Parcel("PKG-102", "Meera"));

for (Parcel parcel : parcels) {
    System.out.println(parcel);
}
```

TRACE THE STATE

After the first add: [PKG-101]

After the second add: [PKG-101, PKG-103]

After insertion at index 1: [PKG-101, PKG-102, PKG-103]

Valid insertion indexes range from 0 through `size()`.

Insertion changes later indexes, so trace the list after every structural modification.

ArrayList operation contracts

Each method has a specific input, return value and possible state change.

METHOD	PURPOSE	RETURN	IMPORTANT CONDITION
add(value)	append element	boolean	element type must match
add(index,value)	insert at position	void	0 through size
get(index)	read element	element	0 through size-1
set(index,value)	replace element	old element	existing index required
remove(index)	remove by position	removed element	later indexes shift left
size()	current element count	int	not the last index
isEmpty()	test size equals zero	boolean	safe before get(0)

For a non-empty list, the last valid access index is `size() - 1`.

Reading and updating by index



get reads a reference; set replaces the reference stored at an existing position.

UNIFLOW EXAMPLE

```
List<Student> students = new ArrayList<>();
students.add(new Student("101", "Asha", "CSE"));
students.add(new Student("102", "Bala", "ECE"));

Student first = students.get(0);
System.out.println(first.getName());

Student old = students.set(1,
    new Student("102", "Bala", "CSE"));
System.out.println(old.getDepartment());
```

EXPRESSION	RESULT
<code>students.size()</code>	2
<code>students.get(0)</code>	Asha object
<code>set(1,...)</code>	returns old Bala object
new index 1 dept	CSE

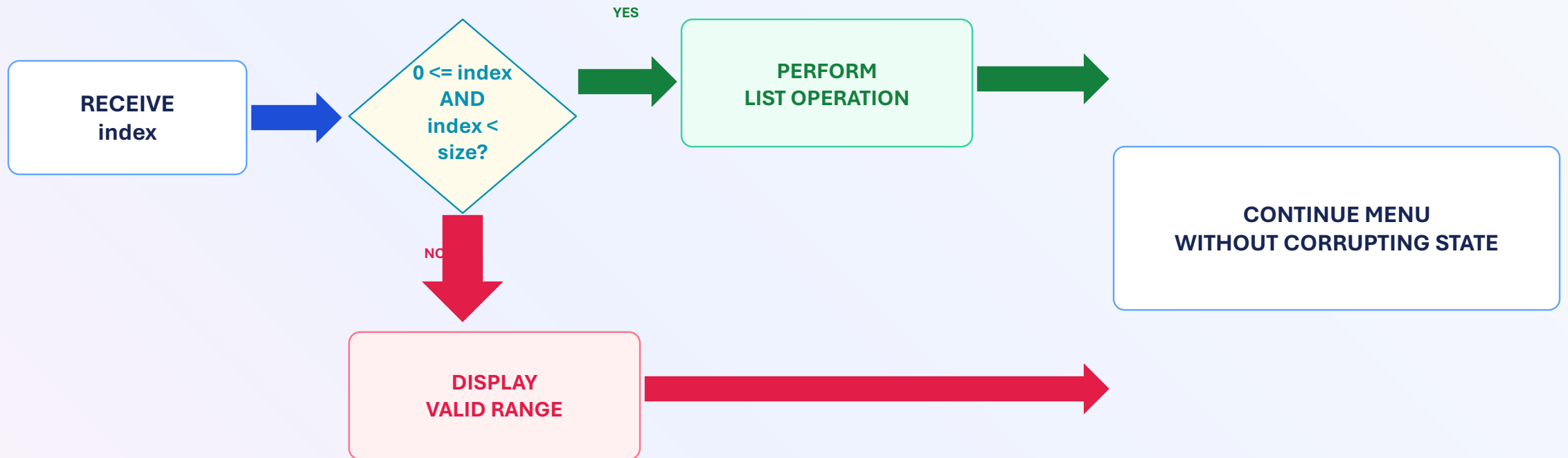
OBJECT REFERENCE

get does not copy the Student automatically. It returns the reference stored in the list.

Use set for replacement; call a validated mutator when only one field should change.

Safe indexed access

Check the index before calling get, set or remove.



Every decision outcome reaches a complete next step; invalid input never reaches the indexed operation.

Removing by index and by object



The overloaded remove methods have different parameters and different return values.

TWO REMOVE FORMS

```
Parcel removed = parcels.remove(1);
System.out.println("Removed: " + removed.getId());

boolean changed = parcels.remove(targetParcel);
System.out.println("Object found and removed: " + changed);

// For List<Integer>, remove(1) means index 1.
// remove(Integer.valueOf(1)) means value 1.
```

WHY BEGINNERS GET CONFUSED

remove(int) selects a position. remove(Object) searches using equality. After a successful removal, every later element shifts one index left.

Choose the remove overload deliberately and trace the resulting indexes.

Searching a list with contains and indexOf



Both methods use equality, so domain-class equality determines whether a logically matching object is found.

LOOKUP EXAMPLE

```
Parcel key = new Parcel("PKG-102", "Meera");

boolean present = parcels.contains(key);
int index = parcels.indexOf(key);

if (index >= 0) {
    System.out.println(parcels.get(index));
} else {
    System.out.println("Parcel not found");
}
```

METHOD	NOT FOUND	FOUND
contains	false	true
indexOf	-1	first matching index
get	do not call	safe with valid index

CO4 CONNECTION

Override equals consistently when two objects with the same business identifier should count as equal.

Never call get(indexOf(...)) until you confirm that the returned index is not -1.

ArrayList strengths and usage boundaries

ArrayList fits ordered, index-based work; another interface may express uniqueness or key lookup more directly.

GOOD FIT

- Dynamic ordered register
- Frequent traversal
- Index-based display or update
- Duplicates allowed by the requirement
- Examples: admission list, parcel arrival list

REQUIREMENT

RECONSIDER

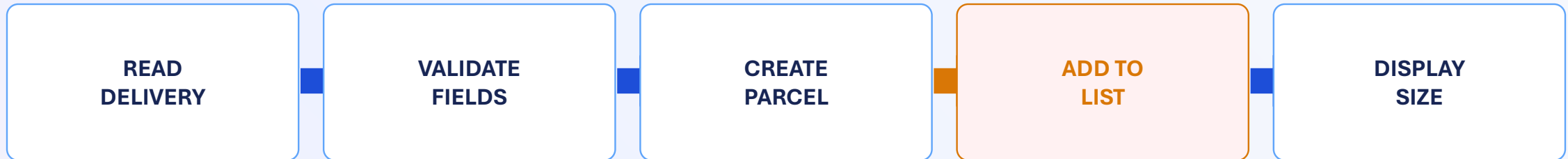
- Duplicate values must be prevented
- Lookup always starts from a unique ID
- First-in service order is the main rule
- Use Set, Map or Queue when its contract matches better

Do not force every data requirement into ArrayList merely because its syntax is familiar.

Smart Locker dynamic parcel workflow



Each successful delivery adds one validated Parcel and the current register remains available for later operations.



CURRENT MILESTONE

The application can admit, display, update and remove parcels without a fixed maximum. Lecture 2 will add a unique-ID directory and a first-in pickup queue.

List size changes only after the complete Parcel passes validation.

Smart Locker list module



Small methods separate admission, display and cancellation operations.

PARCEL REGISTER

```
static void acceptDelivery(List<Parcel> parcels, Parcel p) {
    parcels.add(p);
    System.out.println("Stored at index " + (parcels.size() - 1));
}

static void displayAll(List<Parcel> parcels) {
    if (parcels.isEmpty()) {
        System.out.println("No parcels available");
        return;
    }
    for (Parcel p : parcels) System.out.println(p);
}

static Parcel cancelAt(List<Parcel> parcels, int index) {
    return parcels.remove(index);
}
```

METHOD CONTRACTS

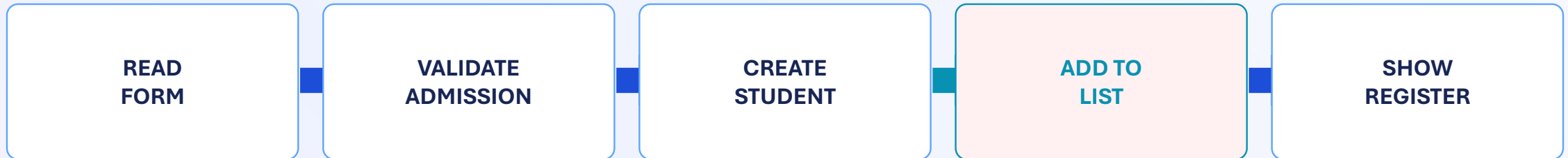
acceptDelivery expects an already validated Parcel. displayAll handles the empty boundary. cancelAt expects a previously checked index and returns the removed object.

Separate methods make each collection operation easier to test and explain.

UniFlow dynamic admission workflow



The same List operations manage Student objects while the application keeps its own validation rules.



PARALLEL DESIGN

Smart Locker stores Parcel objects; UniFlow stores Student objects. The List contract stays the same, while fields and validation remain domain-specific.

Transfer the collection pattern, not the domain fields.

Lecture 1 test and evidence matrix



Normal, boundary and failure tests make list behaviour observable.

TEST	START STATE	ACTION	EXPECTED EVIDENCE
first add	empty list	add one valid object	size 1, index 0
middle insert	three records	add at index 1	later records shift right
replacement	two records	set index 1	size unchanged, old object returned
removal	three records	remove index 0	size 2, later indexes shift left
empty display	empty list	displayAll	clear no-records message
invalid access	size 2	get index 2	blocked before get

Evidence records both the returned value and the collection state after the operation.

Lecture 1 synthesis

ArrayList solves dynamic ordered storage; later lectures add stronger contracts for other data relationships.

STUDENTS CAN NOW

- Choose List for ordered records
- Create typed ArrayList objects
- Add, read, update and remove
- Trace index shifts and returned values
- Guard every indexed operation

BRIDGE

NEXT LECTURE

- Prevent duplicate registrations with Set
- Retrieve records by unique keys with Map
- Display sorted keys with ordered collections
- Process first-in requests with Queue

The best collection is the one whose contract matches the requirement.

Thank you



CO6 Lecture 1 review and exit evidence

SUBMIT THIS EVIDENCE

1. Dynamic Parcel list
2. Dynamic Student list
3. Add, get, set and remove traces
4. Empty-list output
5. Invalid-index guard
6. State-before and state-after table

FINAL VERIFICATION QUESTIONS

Why does List suit an admission register?
What is the last valid access index?
How do set and add differ?
What changes after remove?
Why can contains depend on equals?

LECTURE 2

Set, Map, ordered collections and Queue for uniqueness, lookup, sorted views and service order

Dr. N Chaitanya Kumar

Professor in CSE & Coordinator Freshman Engineering Department

LECTURE 1 COMPLETE: choose, create, modify and verify dynamic lists.